AI для .NET-розробника

Підручник від А до Я

Том 1 · Частина I «Основи» · Розділи 1–3

Навчальний план (зміст усього підручника)

Підручник розрахований на новачка зі стеком .NET (C#, ASP.NET Core, Visual Studio / VS Code / Rider). Кожен том містить 2–3 розділи. Після кожного розділу є практичні завдання.

Том	Розділи	Тема
1 Основи	1	Що таке AI-асистент і велика мовна модель (LLM)
	2	Словник: токени, контекст, промпт, temperature, галюцинації, RAG тощо
	3	Ландшафт інструментів: Copilot, ChatGPT, Claude, Gemini, Cursor, моделі та тарифи
2 Промпти й Copilot	4	Промпт-інжиніринг: як формулювати запити для коду
	5	GitHub Copilot у Visual Studio / VS Code для .NET: автодоповнення, Chat, режими Ask / Edit / Agent
	6	Керування контекстом: #file, @workspace, #codebase, історія чату
3 Налаштування й файли	7	MD-файли інструкцій: `copilot-instructions.md`, `AGENTS.md`, `CLAUDE.md`, `*.instructions.md`
	8	Prompt files (`*.prompt.md`) і Skills (`SKILL.md`)
	9	Custom agents (`*.agent.md`), режими, hooks, дозволи
4 Агенти й MCP	10	MCP (Model Context Protocol): сервери, інструменти, безпека
	11	Агентна робота: tool calling, Copilot CLI, автономність, субагенти
	12	Cloud agent: задачі через issue, PR, CI
5 Практика .NET	13	Тести, рефакторинг, міграції, документація з AI
	14	Рев'ю коду, безпека, ліцензії, приватність
	15	Свої AI-застосунки на .NET: Microsoft.Extensions.AI, Azure OpenAI, Semantic Kernel
6	Додатки	Глосарій, шпаргалка, типові помилки, чекліст проєкту

> **Як працювати:** читайте том, виконуйте завдання на власному невеликому .NET-проєкті (наприклад, Web API «список справ»). Після тому напишіть «наступний том» — і ми продовжимо.

Розділ 1. Що таке AI-асистент і LLM

1.1. Коротко про суть

LLM (Large Language Model, велика мовна модель) — нейромережа, навчена на величезних обсягах тексту і коду. Її базова задача: *передбачити наступний фрагмент тексту (токен)* з огляду на попередній. Звідси здатність писати код, пояснювати, перекладати, підсумовувати.

AI-асистент — програма-обгортка навколо LLM: вікно чату, інтеграція в IDE, доступ до ваших файлів, інструменти (термінал, пошук, запуск тестів). GitHub Copilot, ChatGPT, Claude Code — це асистенти, а GPT, Claude, Gemini — моделі всередині.

Рівень	Що це	Приклад
Модель	«Мозок», що генерує текст	GPT-5, Claude Sonnet, Gemini
Асистент / продукт	Інтерфейс + інструменти + налаштування	GitHub Copilot, ChatGPT, Cursor
Ваші налаштування	Інструкції, skills, MCP для вашого проєкту	`copilot-instructions.md`

1.2. Як це працює (спрощено)

1. Ваш текст розбивається на **токени** (шматки слів).
2. Модель на основі токенів обчислює ймовірності наступного токена і вибирає один.
3. Процес повторюється, поки відповідь не завершена.

Важливо: модель **не «знає» факти, як база даних**. Вона відтворює правдоподібні патерни. Тому вона може упевнено помилятися — це називають **галюцинацією**.

1.3. Три покоління роботи з AI

Режим	Опис	Приклад у .NET
Автодоповнення	Підказує наступний рядок/блок під час набору	Пишете `public async Task<` — Copilot пропонує продовження
Чат (Ask)	Ви ставите питання, отримуєте відповідь	«Поясни, чому тут NullReferenceException»
Агент (Agent)	Сам читає файли, редагує, запускає `dotnet build/test`, виправляє помилки	«Додай ендпоінт DELETE /todos/{id} і тест до нього»

1.4. Що AI робить добре, а що погано

Добре	Погано / ризиковано
Шаблонний код (CRUD, DTO, мапінг)	Нові/рідкісні API та версії бібліотек (може вигадати метод)
Юніт-тести, регулярні вирази, SQL/LINQ	Складна архітектурна відповідальність без чіткого контексту
Пояснення чужого коду, рефакторинг	Безпека: може згенерувати вразливий код
Документація, коментарі, commit-повідомлення	Точні обчислення, свіжі новини без пошуку

> **Золоте правило:** AI — це швидкий молодший колега-стажист. Ви відповідаєте за результат: завжди читайте, збирайте (`dotnet build`) і тестуйте (`dotnet test`) те, що він написав.

> **Завдання 1.**
>
> 1. Створіть проєкт: `dotnet new webapi -n TodoApi` .
> 2. Попросіть будь-який чат: «Поясни структуру цього проєкту для новачка» і вставте вміст `Program.cs` .
> 3. Знайдіть у відповіді хоча б одну неточність або спрощення. Це тренує критичне мислення.

Розділ 2. Словник: терміни, які потрібно знати

Це найважливіший розділ-довідник. Повернутися до нього можна в будь-який момент; повний глосарій буде в додатках.

2.1. Базові поняття моделі

Термін	Пояснення
Токен	Одиниця тексту для моделі (≈ 3–4 англ. символи; українська й код «дорожчі» — більше токенів). Ціна й ліміти рахуються в токенах.
Контекстне вікно (context window)	Максимум токенів, які модель «бачить» за раз: ваш запит + історія + файли + відповідь. Вийшло за межу — старе забувається або обрізається.
Промпт (prompt)	Вхідний текст-запит до моделі.
Системний промпт (system prompt)	Приховані/базові інструкції, що задають роль і правила моделі («Ти асистент-програміст…»). Файли інструкцій розширюють саме цей рівень.
Completion / відповідь	Те, що згенерувала модель.
Temperature	«Креативність»: 0 — передбачувано й стабільно, вище — різноманітніше. Для коду зазвичай низька.
Top-p / top-k	Інші способи обмежити вибір наступного токена.
Max tokens	Ліміт довжини відповіді.
Галюцинація	Правдоподібна, але хибна відповідь (вигаданий метод `List.RemoveDuplicates()`).
Knowledge cutoff	Дата, до якої модель мала навчальні дані. Нові версії .NET можуть бути їй невідомі.
Мультимодальність	Модель працює не лише з текстом, а й із зображеннями (скріншот UI → код).
Reasoning / «thinking» модель	Модель, що «міркує» перед відповіддю: повільніша й дорожча, але краща для складних задач.
Fine-tuning	Донавчання моделі на своїх даних. Для звичайного розробника зазвичай не потрібне.
Embedding	Числовий вектор, що представляє зміст тексту; основа семантичного пошуку.

2.2. Терміни роботи з контекстом і знаннями

Термін	Пояснення
Контекст (context)	Усе, що ви надали моделі: файли, виділений код, історія чату, інструкції.
Grounding	«Заземлення» відповіді на реальних даних (ваш код, документація), щоб зменшити вигадки.
RAG (Retrieval-Augmented Generation)	Спершу знайти релевантні фрагменти (пошук по embeddings), потім додати їх у промпт.
Vector database	Сховище embeddings для семантичного пошуку.
Chunking	Розбиття великих документів на шматки для індексації.
Compaction / summarization	Стиснення довгої історії в короткий підсумок, коли контекст майже заповнений.

2.3. Терміни промптингу

Термін	Пояснення
Zero-shot	Запит без прикладів.
Few-shot	Запит з 1–3 прикладами очікуваного формату.
Chain-of-thought	Прохання міркувати крок за кроком.
Role prompting	«Ти — досвідчений .NET-архітектор…»
Structured output	Відповідь у суворому форматі (JSON за схемою).
Prompt injection	Атака: шкідливий текст у даних (файл, веб-сторінка) намагається «перепрограмувати» модель. Дуже важливо для агентів.

2.4. Терміни агентів і екосистеми (огляд — детально в розділах 7–12)

Термін	Пояснення
Agent (агент)	LLM, що в циклі «думає → викликає інструмент → дивиться результат → продовжує», доки задача не виконана.
Tool / function calling	Механізм, у якому модель просить виконати функцію (прочитати файл, `dotnet test`), а програма виконує й повертає результат.
MCP (Model Context Protocol)	Відкритий стандарт підключення зовнішніх інструментів і даних (GitHub, БД, браузер) до будь-якого AI-клієнта. «USB-C для AI».
Custom instructions	Постійні правила для проєкту в `.md`-файлах (стиль коду, команди збірки).
Skill	Папка з `SKILL.md` і ресурсами — «навичка», яку агент підвантажує, коли задача підходить за описом.
Prompt file	Збережений багаторазовий промпт (`*.prompt.md`), що викликається командою `/назва`.
Custom agent / chat mode	Персона з власними інструкціями й набором дозволених інструментів (рев'юер, архітектор).
Субагент (subagent)	Допоміжний агент, якому основний делегує підзадачу в окремому контексті.
Hooks	Скрипти, що спрацьовують у певні моменти (перед запуском інструмента, після редагування) — для контролю й автоматизації.
Human-in-the-loop	Людина підтверджує небезпечні дії (запуск команд, запис файлів).
Autopilot / auto-approve	Агент виконує дії без запитів. Швидко, але ризиковано.
Premium request / квота	Облікова одиниця у Copilot: складні моделі й агентні режими споживають більше.

> **Практичний висновок:** якість відповіді = якість моделі × якість контексту × якість інструкцій. Далі в підручнику ми вчимося керувати двома останніми множниками.

> **Завдання 2.** Без підглядання поясніть своїми словами: чим відрізняється токен від слова; чому довга розмова може «забути» початок; чим skill відрізняється від MCP-сервера (підказка: skill — знання/інструкція, MCP — доступ до зовнішніх дій і даних).

Розділ 3. Ландшафт інструментів

Примітка: ринок змінюється щомісяця; назви моделей і тарифи уточнюйте на офіційних сайтах.

3.1. Типи інструментів

Тип	Приклади	Коли використовувати
Асистент у IDE	GitHub Copilot (Visual Studio, VS Code, Rider), JetBrains AI Assistant, Cursor	Щоденне кодування
Веб-чат	ChatGPT, Claude, Gemini, Microsoft Copilot	Питання, навчання, чернетки, аналіз
Термінальні агенти	GitHub Copilot CLI, Claude Code, Codex CLI	Багатокрокові задачі над репозиторієм
Хмарні агенти	Copilot cloud agent (issue → PR), Codex cloud	Фонові задачі, що закінчуються Pull Request
API / SDK	Azure OpenAI, OpenAI, Anthropic, Microsoft.Extensions.AI	Вбудовування AI у власний застосунок

3.2. Що обрати .NET-розробнику-новачку

- **Основний інструмент:** GitHub Copilot — глибока інтеграція з Visual Studio і VS Code, підтримка кількох моделей, агентний режим, MCP, інструкції та skills.
- **Для навчання й консультацій:** один веб-чат (ChatGPT / Claude / Gemini) на ваш смак.
- **Пізніше:** Copilot CLI та cloud agent для автоматизації.

3.3. Різні моделі — різні сильні сторони

Клас моделі	Характеристика	Використання
Швидкі / «mini», «flash», «haiku»	Дешеві, швидкі, простіші	Автодоповнення, дрібні правки, пояснення
Універсальні «стандартні»	Баланс ціни й якості	Основна робота, агентні задачі
Reasoning / «max»	Повільні, дорогі, глибше міркують	Складні баги, архітектура, великі рефакторинги

Правило: починайте зі стандартної моделі, ускладнюйте лише коли вона не справляється. У Copilot модель обирається у випадаючому списку чату; режим **Auto** підбирає її за вас.

3.4. Ваш перший робочий цикл у Copilot

1. Встановіть Visual Studio 2022+ (або VS Code з C# Dev Kit) і увійдіть під GitHub-акаунтом із Copilot.
2. Відкрийте `TodoApi`. Відкрийте вікно **Copilot Chat**.
3. Виділіть код у `Program.cs` і спитайте: «Explain».
4. Попросіть: «Додай модель `TodoItem` (Id, Title, IsDone) та мінімальні ендпоінти CRUD у пам'яті».
5. Перевірте: `dotnet build`, запустіть, випробуйте через Swagger/.http-файл.

3.5. Безпека з першого дня

- Не вставляйте в чат паролі, ключі API, рядки підключення, персональні дані.
- Секрети тримайте в User Secrets / Key Vault, не в коді — AI має це знати (пропишемо в інструкціях, розділ 7).
- Перевіряйте, чи дозволяє ваш роботодавець використання AI з кодом проєкту.
- Перевіряйте залежності, які пропонує AI: пакет має існувати у NuGet і бути надійним.

> **Завдання 3.**
>
> 1. Виконайте цикл із 3.4 на `TodoApi`.
> 2. Зробіть одне й те саме завдання двома різними моделями (або чатами) і порівняйте код та пояснення.
> 3. Запишіть у нотатки: які 3 терміни з розділу 2 були для вас новими.

> **Далі — Том 2:** розділи 4–6 — як правильно формулювати запити, працювати з Copilot у Visual Studio та керувати контекстом.